

Graph Database Cloud Benchmark: CognoDB vs. Neo4j, Memgraph, ArangoDB, Apache AGE

A reproducible benchmark comparing **CognoDB Cloud** against four self-hosted graph databases on identical hardware limits, an identical dataset, and identical query workloads.

TL;DR: Memgraph is fastest on raw query latency by a wide margin (sub-millisecond reads), Neo4j is close behind with occasional GC-driven latency spikes, ArangoDB trades point-lookup speed for consistent ~50ms AQL overhead on everything else, and CognoDB while competitive on raw engine performance is dominated by network round-trip latency since it's the only platform benchmarked over the internet rather than locally. Apache AGE was excluded after its ingest throughput proved ~30-50x slower than the other platforms; see [Caveats](#).

Quick Start

Follow these steps in order. Total setup time: ~15-20 minutes plus however long you let the data load (a few minutes for 4 of the 5 platforms; see the Apache AGE caveat below for why it's excluded).

1. Clone the repo and set up Python

```
git clone <your-repo-url>
cd graph-db-benchmark
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

2. Set up your CognoDB Cloud instance

1. Go to <https://console.cognodb.com/signup> and create a free account (no credit card required).
2. Create a free (c0) instance, pick a region.
3. **Copy the connection URI and password immediately** the password is shown exactly once. You'll get:
 - A URI like `bolt+s://<instance-id>.databases.cognodb.cloud`
 - A password for user `cognodb`

3. Configure environment variables

Create and open `.env` file and fill in your values:

```
# --- CognoDB Cloud ---
COGNODB_URI=bolt+s://<your-instance-id>.databases.cognodb.cloud
COGNODB_USER=cognodb
COGNODB_PASSWORD=<the password CognoDB showed you once>

# --- Docker-hosted comparison databases ---
```

```
# These can be any password you choose -- they're only used locally.
NEO4J_PASSWORD=NEO4J123
MEMGRAPH_PASSWORD=MEMGRAPH123
ARANGO_PASSWORD=ARANGO123
AGE_PASSWORD=AGE123

# --- Local connection settings (defaults below match docker-compose.yml) ---
ARANGO_URL=http://localhost:8529
AGE_HOST=localhost
AGE_PORT=5433
AGE_DBNAME=graphbench
AGE_USER=age
```

Only `COGNODB_URI` and `COGNODB_PASSWORD` need real values from your account everything else can be left as shown, since those passwords only apply to your local Docker containers.

4. Start the 4 local comparison databases

```
docker compose up -d
docker compose logs -f          # watch until all 4 report healthy, then Ctrl+C
```

This starts Neo4j, Memgraph, ArangoDB, and Apache AGE, each capped to **0.5 vCPU / 512MB RAM** see [Fairness & Resource Parity](#) for why 512MB and not the 256MB the assignment brief describes.

5. Get the dataset

If you don't already have `data/raw/soc-pokec-relationships.txt.gz` and `data/raw/soc-pokec-profiles.txt.gz`:

```
python data/download_dataset.py
```

Then trim it down to a size that fits every platform's free tier:

```
python data/trim_dataset.py
```

This produces `data/trimmed/edges.csv` and `data/trimmed/profiles.csv` (~11,250 nodes, ~233,000 relationships see [Dataset](#) for methodology).

`data/trimmed/*.csv` is already committed to this repo (it's only ~3.6MB), so you can skip steps 5 and go straight to step 6 unless you want to regenerate it yourself from the full SNAP dataset.

6. Load the data into every database

```
python scripts/load_all.py
```

This loads Neo4j, Memgraph, CognoDB, and ArangoDB, creates indexes on each, and saves ingest-throughput results to `results/load_results.json`.

Apache AGE is intentionally excluded from this step and every step after see [Caveats](#) for why.

7. Run the benchmark workloads

```
python benchmark/runner.py --all
```

This runs all 6 required query workloads (1/2/3-hop traversal, point lookup, filtered lookup, aggregation 150 iterations each after 20 warm-up iterations) plus the concurrent mixed-workload sweep (concurrency 1/10/40, 15 seconds each, 80/20 read/write mix) against every loaded platform. Expect this to take a while CognoDB specifically runs over the network and is noticeably slower per-query.

Results save to `results/<platform>_workload.json` and a merged `results/workload_results.json`.

You can also run one platform at a time:

```
python benchmark/runner.py --platform neo4j
```

8. Generate the results tables

```
python scripts/generate_report.py > results/results_tables.md
```

This is already done for you below the tables in [Results](#) were generated exactly this way.

Fairness & Resource Parity

Every comparison platform is capped to match CognoDB's **actual observed instance specs**, not the specs described in the assignment brief.

Discrepancy note: the assignment brief states CognoDB's free tier as 0.5 vCPU / 256MB RAM / 1GB disk. My provisioned instance's dashboard reported **512MB RAM**. I matched the real, observed spec (512MB) rather than the documented figure, since fairness requires capping every platform to what's actually being benchmarked.

Resource Cap applied to all 5 platforms

Resource	Cap applied to all 5 platforms
vCPU	0.5 (enforced via Docker <code>deploy.resources.limits</code> for the 4 local platforms; CognoDB's free tier is inherently limited to this)

Resource	Cap applied to all 5 platforms
RAM	512MB (enforced via Docker for local platforms; matches CognoDB's actual dashboard spec)
Disk	Not hard-capped by Docker in a portable way. Instead, disk parity is enforced by using the identical ~3.6MB trimmed dataset for every platform, which comfortably fits well under 1GB everywhere.

CognoDB is the one platform that cannot be resource-capped it's a managed cloud service, so I can only observe and match its advertised/observed specs on the other 4 platforms, not impose limits on CognoDB itself. This is inherent to comparing a managed cloud service against self-hosted alternatives; see the Analysis section for how this affects latency interpretation.

Dataset

- **Source:** [SNAP soc-Pokec](#) a directed social network with ~1.6M nodes and ~30.6M relationships.
- **Trimming method:** the full dataset is far too large for a 512MB instance, so I take an **induced subgraph** on the top-degree nodes: rank all nodes by total (in+out) degree, keep the top N, then keep only edges where *both* endpoints are in that set. This keeps the trimmed graph dense and well-connected (as opposed to a random edge sample, which would fragment it) important for multi-hop traversal queries to return meaningful results.
- **Final size:** 11,250 nodes, 233,031 relationships within the assignment's recommended 100k-500k relationship range.
- **Reproducibility:** [data/trimmed/*.csv](#) is committed directly to this repo (~3.6MB total), so anyone cloning it can load the data immediately without downloading the full ~1.6GB SNAP dataset first.
- **Node properties kept:** [public](#), [gender](#), [age](#), [region](#), [completion_percentage](#) (from the Pokec profiles file), used for the filtered-lookup and aggregation workloads. All fields had 0% null rate after trimming.
- **Graph model used across all platforms:** `(:User {node_id, public, gender, age, region, completion_percentage})-[:FRIEND]->(:User)`.

Trade-off worth noting: selecting top-degree nodes maximizes connectivity for meaningful traversal results, but also means the subgraph is a dense "core" 3-hop path *counts* between hub nodes can explode combinatorially. This shows up directly in the results below (see 3-Hop Traversal Latency) and is discussed in [Analysis](#).

Results

Data Loading (Ingest Throughput)

Platform	Nodes	Relationships	Load Time (s)	Nodes/sec	Rels/sec
cognodb	11,250	233,031	119.95	93.79	1,942.73
neo4j	11,250	233,031	49.75	226.15	4,684.35
memgraph	11,250	233,031	7.24	1,553.45	32,178.06
arangodb	11,250	233,031	28.36	396.62	8,215.54

1-Hop Traversal Latency (p50 / p95, ms)

Platform	p50 (ms)	p95 (ms)	mean (ms)	samples	failed
cognodb	129.93	131.18	130.09	150	0
neo4j	1.23	2.35	2.06	150	0
memgraph	1.03	1.45	1.08	150	0
arangodb	50.02	59.44	50.53	150	0

2-Hop Traversal Latency (p50 / p95, ms)

Platform	p50 (ms)	p95 (ms)	mean (ms)	samples	failed
cognodb	132.56	153.47	135.79	150	0
neo4j	1.60	2.92	1.99	150	0
memgraph	1.11	1.95	1.23	150	0
arangodb	50.84	62.10	52.16	150	0

3-Hop Traversal Latency (p50 / p95, ms)

Platform	p50 (ms)	p95 (ms)	mean (ms)	samples	failed
cognodb	176.84	1,965.05	605.16	147	3
neo4j	4.64	73.78	14.62	150	0
memgraph	3.20	48.59	10.99	150	0
arangodb	91.82	957.27	270.11	150	0

Point Lookup Latency (p50 / p95, ms)

Platform	p50 (ms)	p95 (ms)	mean (ms)	samples	failed
cognodb	129.43	132.56	129.80	150	0
neo4j	1.71	3.33	2.44	150	0
memgraph	1.12	1.57	1.24	150	0
arangodb	1.38	2.03	1.46	150	0

Filtered/Indexed Lookup Latency (gender=1, age>=25) (p50 / p95, ms)

Indexed fields per platform: Neo4j / Memgraph / CognitoDB `node_id`, `gender`, `age` (native property indexes). ArangoDB `_key` (= `node_id`, primary index) plus persistent indexes on `gender`, `age`.

Platform	p50 (ms)	p95 (ms)	mean (ms)	samples	failed
cognodb	140.75	144.60	140.94	150	0
neo4j	3.52	5.89	3.79	150	0
memgraph	2.69	3.59	2.60	150	0
arangodb	50.01	59.72	50.60	150	0

Aggregation Latency (count by gender)

Platform	p50 (ms)	p95 (ms)	mean (ms)	samples	failed
cognodb	146.30	150.05	147.24	150	0
neo4j	5.76	45.84	11.07	150	0
memgraph	3.86	19.72	5.64	150	0
arangodb	50.05	58.43	50.59	150	0

Concurrent Mixed Read/Write Throughput (80% read / 20% write)

Platform	Concurrency	Throughput (ops/sec)	Total Ops	Errors
cognodb	1	7.33	110	0
cognodb	10	71.93	1,079	0
cognodb	40	230.13	3,452	17
neo4j	1	600.80	9,012	0
neo4j	10	628.33	9,425	0
neo4j	40	683.07	10,246	0
memgraph	1	912.87	13,693	0
memgraph	10	1,070.80	16,062	2
memgraph	40	227.67	3,415	7
arangodb	1	71.47	1,072	0
arangodb	10	431.13	6,467	0
arangodb	40	457.47	6,862	0

Footprint

Platform	Details
cognodb	not observable (free-tier console has no storage/memory introspection endpoint)
neo4j	not observable (APOC plugin not installed on the community image used)

Platform	Details
memgraph	11,250 vertices, 233,031 edges (self-reported, matches load target exactly), memory_res 182.35MiB, peak 193.44MiB, disk_usage 72.42MiB, allocation_limit 450MiB, storage_mode IN_MEMORY_TRANSACTIONAL
arangodb	not observable

Analysis

Memgraph wins on every latency metric, often by an order of magnitude (sub-millisecond p50 on 1-hop, 2-hop, and point lookup). Makes sense it's an in-memory C++ engine built for exactly this, versus Neo4j's JVM or ArangoDB's HTTP+AQL model.

Neo4j sits close to Memgraph on median latency but has a much wider p50-to-p95 gap (point lookup: 1.71ms median, 3.33ms p95; aggregation: 5.76ms vs 45.84ms). Memgraph doesn't show this spread on the same queries.

ArangoDB splits sharply between point lookup (1.38ms) and everything else (~50ms flat, whether it's a 1-hop traversal or a full aggregation). Point lookup skips AQL entirely via direct document GET; everything else pays AQL parse/plan overhead over HTTP, and that fixed cost dominates regardless of how much work the query actually does.

CognoDB sits at a ~130ms floor on every query, including trivial point lookups. It's the only platform benchmarked over the internet instead of localhost, so this looks like network + TLS round-trip cost rather than query engine performance.

3-hop traversal has a rough tail on every platform (CognoDB p95 1.97s, ArangoDB p95 957ms, even Memgraph and Neo4j show their widest p50-p95 gaps here). This traces back to the dataset, not the platforms: `MATCH (a)-[:FRIEND*3]->(b)` counts distinct paths, not distinct destinations, and the trimmed dataset deliberately keeps high-degree hub nodes for connectivity so path counts between hubs can blow up combinatorially for a handful of unlucky start nodes. Every platform paid this cost roughly equally.

Mixed-workload throughput drops hard at concurrency=40 for Memgraph (1,070 → 227 ops/sec) and CognoDB (errors appear, though ops/sec still climbs), while Neo4j and ArangoDB scale up cleanly with zero errors at any concurrency. Memgraph and CognoDB both threw transient conflict errors under high write concurrency (`Cannot resolve conflicting transactions`, `DeadlockDetected`) optimistic concurrency control contending on the same small set of frequently-written nodes.

Caveats

Honest record of everything that didn't go perfectly, per the assignment's explicit request to document not hide methodology issues.

- **CognoDB's actual RAM (512MB) differs from the assignment brief's stated 256MB.** I capped all 4 comparison platforms to 512MB to match the real observed instance. See [Fairness & Resource Parity](#).
- **CognoDB runs over the public internet; the other 4 run in local Docker containers.** Its latency numbers include real network round-trip time the others don't pay, so comparisons aren't purely

engine-vs-engine. A fully controlled test would need all 5 platforms deployed to cloud instances in the same region outside this assignment's 48-hour scope.

- **Apache AGE is excluded from the workload benchmark.** Its edge-loading throughput measured ~32 edges/sec 30-50x slower than the other platforms so the full load was stopped before completion to preserve the time budget. Likely cause: AGE doesn't push Cypher property-equality **MATCH** down to the Postgres btree indexes I created, forcing full-table scans during edge creation. The adapter ([adapters/age_db.py](#)) is complete and documents this, but wasn't benchmarked on incomplete data.
- **3 of 150 CognitoDB 3-hop iterations failed** with connection-drop errors rather than a query timeout likely the same dense-hub-node path explosion (see Analysis) triggering a free-tier network/proxy reset rather than an in-band error. The driver recovered automatically and all later workloads ran normally.
- **Footprint is "not observable" for CognitoDB** (no console API), **Neo4j** (APOC not installed on the community image), and **ArangoDB** (statistics endpoint response shape didn't match expectations; not pursued further). Memgraph's footprint is fully reported via **SHOW STORAGE INFO**.
- **Neo4j load batch sizes needed tuning down** (1,000/5,000 → 300/500 rows/transaction) after **MemoryPoolOutOfMemoryError** during loading and clearing dense batches (avg degree ~41) exceeded the transaction memory pool under the resource cap. Smaller batches fixed it without raising the memory cap itself.
- **Index-creation order caused a real bug:** indexes were originally created *after* loading, so edge creation ran as an unindexed full scan ($O(\text{edges} \times \text{nodes})$) and effectively hung. Fixed by indexing **node_id** right after nodes load, before edges see [adapters/cypher_base.py](#) and [adapters/age_db.py](#).

Repository Structure

```
graph-db-benchmark/
├── README.md                # this file
├── requirements.txt
├── docker-compose.yml      # Neo4j, Memgraph, ArangoDB, Apache AGE
├── .env.example
├── data/
│   ├── download_dataset.py  # downloads SNAP soc-Pokec to data/raw/
│   ├── trim_dataset.py     # trims to data/trimmed/ (~11k nodes, ~233k edges)
│   └── raw/                # gitignored -- full SNAP files (download
separately)
│   └── trimmed/            # COMMITTED -- edges.csv, profiles.csv,
node_id_mapping.csv, summary.json (~3.6MB)
├── adapters/
│   ├── base.py             # abstract interface every adapter implements
│   ├── cypher_base.py      # shared Cypher/Bolt implementation
│   ├── neo4j_db.py
│   ├── memgraph_db.py
│   ├── cognodb.py
│   ├── arangodb_db.py      # AQL-based
│   └── age_db.py           # Apache AGE (Cypher-via-SQL) -- see Caveats
└── scripts/
```



```
|   |─ load_all.py           # loads data into every registered adapter
|   |─ check_age_progress.py # standalone progress checker (debugging aid)
|   └─ generate_report.py    # JSON results -> markdown tables
|─ benchmark/
|   └─ runner.py             # runs the full section 5.2 workload suite
|─ test/
|   |─ test_adaptor.py       # test if adaptor works fine in uploading data,
creating index, etc
|   └─ test_cognoDB.py       # check if it connect to running instance
correctly or not
└─ results/
    |─ load_results.json
    |─ workload_results.json
    |─ <platform>_workload.json
    └─ results_tables.md
```

Reproducing These Exact Results

1. Follow [Quick Start](#) steps 1-6.
2. `python benchmark/runner.py --all`
3. `python scripts/generate_report.py > results/results_tables.md`

All randomness (node sampling for traversal/lookup queries) uses each platform's native random-sampling query, so exact per-run numbers will vary slightly, but percentile latencies and overall platform rankings should reproduce consistently.